

Name: Kassim Musa Abass  
Reg: SPH32/3338/2025  
Program: Embedded Systems Intro

## **Week 7 & 8: ESP32 Traffic Light & Serial Communication**

### **Objective:**

To configure and program the NodeMCU ESP32 using the Arduino IDE, build a traffic light system, and establish UART serial communication to transmit the system's state to a Raspberry Pi Pico.

### **Introduction:**

In complex embedded systems, multiple microcontrollers often need to communicate. This project builds a traffic light controller on the ESP32 and uses the UART (Universal Asynchronous Receiver-Transmitter) protocol to stream state changes to a Raspberry Pi Pico over a two-wire connection (TX/RX).

### **1. Theoretical Concepts**

UART Serial Communication: UART is a hardware communication protocol that uses asynchronous serial communication with configurable speeds (baud rates). In our setup, the ESP32 acts as the transmitter (TX) and the Pico as the receiver (RX) running at 9600 baud.

Ground Loop: A critical aspect of serial communication between two discrete boards is ensuring they share a common Ground (GND). Without a common ground reference, the voltages representing 1s and 0s become erratic, resulting in corrupted data.

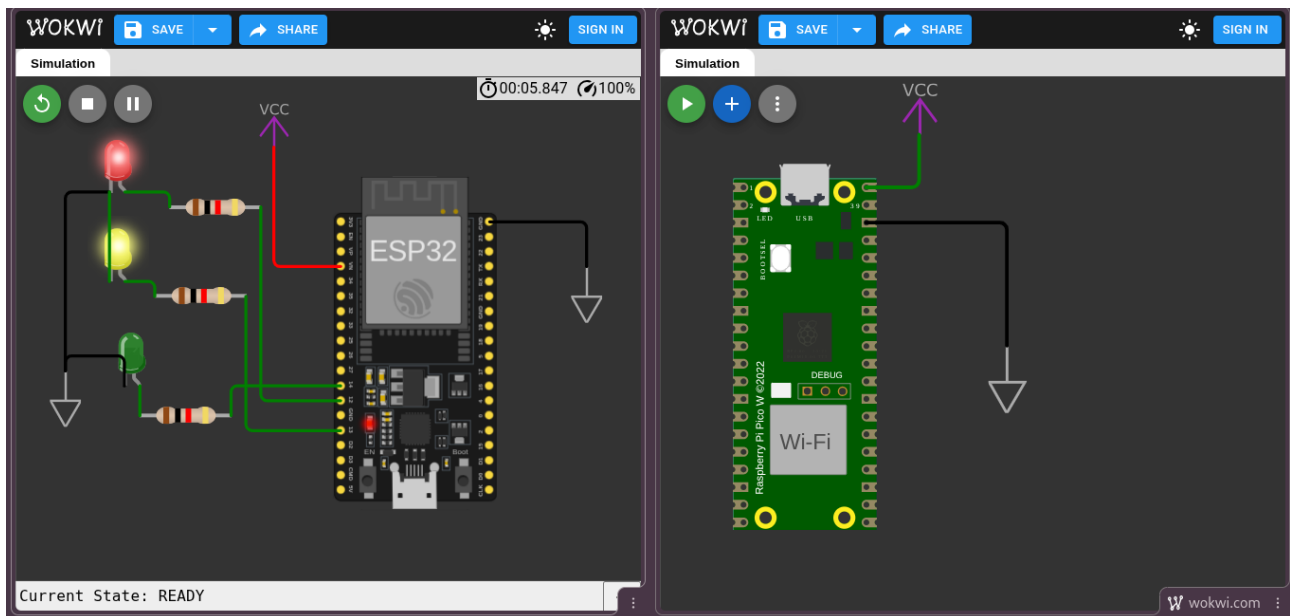
### **2. Source Code - ESP32 (Transmitter)**

```
1 // esp32_traffic_serial.ino
2 // Week 7 & 8: ESP32 Traffic Light Controller & Serial TX
3 // This code cycles through a standard traffic light sequence and send
4 // via UART to another microcontroller (e.g., Raspberry Pi Pico).
5
6 const int RED_PIN = 12;
7 const int YELLOW_PIN = 13;
8 const int GREEN_PIN = 14;
9
10 void setup() {
11     pinMode(RED_PIN, OUTPUT);
12     pinMode(YELLOW_PIN, OUTPUT);
13     pinMode(GREEN_PIN, OUTPUT);
14
15     // Initialize standard Serial for debugging
16     Serial.begin(115200);
17
18     // ESP32 Serial2 default pins: TX2 = GPIO 17, RX2 = GPIO 16
19     // This is used for UART communication with the Pico
20     Serial2.begin(9600);
21
22     Serial.println("ESP32 Traffic Light & Serial TX initialized.");
23 }
24
25 void setLights(bool red, bool yellow, bool green, const char* stateName) {
26     digitalWrite(RED_PIN, red ? HIGH : LOW);
27     digitalWrite(YELLOW_PIN, yellow ? HIGH : LOW);
28     digitalWrite(GREEN_PIN, green ? HIGH : LOW);
29
30     // Send the state over UART to the Pico
```

### 3. Source Code - RPi Pico (Receiver)

```
sketch.ino • diagram.json • Library Manager
1 // pico_receiver.ino
2 // Week 7 & 8: RPi Pico Serial RX
3 // This code receives the traffic light state from the ESP32 via UART and prints it.
4
5 // For RPi Pico, Serial1 uses GP0 (TX) and GP1 (RX) by default.
6 void setup() {
7   // Initialize USB serial for debugging
8   Serial.begin(115200);
9
10  // Initialize UART communication to receive from ESP32.
11  // Wiring: ESP32 TX (GPIO 17) -> Pico RX (GP1). Must share common GND.
12  Serial1.begin(9600);
13
14  Serial.println("Pico Serial Receiver initialized.");
15  Serial.println("Waiting for traffic light states from ESP32...");
16 }
17
18 void loop() {
19   // Check if data is available from the ESP32 over UART
20   if (Serial1.available()) {
21     String stateMessage = Serial1.readStringUntil('\n');
22     stateMessage.trim(); // Remove whitespace/newlines
23
24     if (stateMessage.length() > 0) {
25       Serial.print("[From ESP32 via UART]: Traffic Light is now -> ");
26       Serial.println(stateMessage);
27
28       // Additional logic can be added here, such as:
29       // if (stateMessage == "STOP") { digitalWrite(PEDESTRIAN_LIGHT, HIGH); }
30     }
31   }
32 }
```

#### 4. Hardware Integration (Serial Communication)



#### 5. Reflection

A significant learning moment was understanding how to use HardwareSerial (Serial1/Serial2) on the ESP32 and Pico to communicate between devices while reserving the main 'Serial' bus for PC debugging. Verifying that the logic level voltages (3.3V) were compatible between the ESP32 and the Pico was also a critical step before making the physical connections.